FocusLock: Engineering Guidebook

Webcam-Based Gaze Control System — Step-by-Step Architecture & Theory

1. Executive Overview & Core Philosophy

FocusLock is an educational computer-vision project designed to achieve hands-free computer interaction using standard webcam hardware. Unlike commercial eye-trackers (which rely on infrared illumination and specialized cameras), FocusLock uses standard RGB imagery processed through a structured 6-layer engineering pipeline.

> ***Crucial College Demo Concept:*** *MediaPipe is only the sensor. The true novelty and engineering value of this project lies in the downstream stages:* ***Coordinate Normalization****,* ***Temporal Filtering (Jitter Reduction)****,* ***5-Point Linear Calibration Mapping****, and* ***Dwell-Based Intent Detection****.*

2. Full Architectural Pipeline

Layer	Module	Primary Function	Key Math / Technique
1. Sensing	iris_tracker.py	Extract 478 face & iris landmarks	MediaPipe Tasks FaceLandmarker (RGB)
2. Normalization	gaze_estimator.py	Make iris position relative to eye socket	Ratio: (iris - min) / (max - min)
3. Filtering	moving_average.py	Eliminate sensor noise & micro-saccades	Sliding Window FIFO Mean (N=7)
4. Calibration	calibrator.py	Map normalized gaze to screen coordinates	1D Linear Least Squares (numpy.polyfit)
5. Intent	dwell_detector.py	Prevent unintended clicks (Midas Touch)	Continuous Zone Dwell Timer (1.8s)
6. Actuation	presentation_controller.py	Simulate presentation keypresses	PyAutoGUI Left/Right Arrow Dispatch

3. Stage 1: Sensing & Iris Extraction

Objective: Capture live webcam video and locate anatomical facial features without training custom neural networks.

Detailed Technical Decisions:

- **Modern MediaPipe Tasks API:** Uses FaceLandmarker with face_landmarker.task model (~30MB), providing 478 3-D landmarks.
- **Iris Center Indices:** Landmark **468** represents the right iris center and landmark **473** represents the left iris center (in anatomical orientation).
- **BGR to RGB Conversion:** OpenCV decodes camera streams into BGR format, whereas MediaPipe models are trained on RGB tensors. cv2.cvtColor() is strictly mandatory.
- **Mirror Flipping:** We apply cv2.flip(frame, 1) so that moving your head to the right mirrors user intuition in the GUI.

4. Stage 2: Gaze Normalization & Signal Smoothing

Objective: Transform raw pixel coordinates into a scale-invariant ratio within the eye region, and eliminate high-frequency tremor.

The Mathematical Formulation:

1. **Eye Bounding Box Extraction:** We determine the minimum and maximum pixel boundaries of the eyelids:
   W_eye = X_max - X_min, H_eye = Y_max - Y_min

2. **Normalization Formula:**
   Gaze_X = (Iris_X - X_min) / W_eye
   Gaze_Y = (Iris_Y - Y_min) / H_eye
This maps the iris to a dimensionless coordinate in [0.0, 1.0]. When your head shifts relative to the camera, both the eye bounding box and the iris shift concurrently, making the ratio partially head-pose invariant.

3. **Dual-Eye Fusion:** Gaze coordinates are estimated for both the left and right eyes independently and averaged: Gaze = (Gaze_Left + Gaze_Right) / 2.0. This halves single-eye landmark variance.

Noise & The Moving-Average Filter:

Webcam gaze data suffers from *high-frequency jitter* due to optical sensor noise, lighting variations, and involuntary biological eye movements (microsaccades).
We implement a sliding window filter using collections.deque(maxlen=7):
   Filtered_Gaze(t) = (1 / N) * SUM(Raw_Gaze(t - i)) for i in [0..N-1]

> ***Engineering Trade-off:*** *A larger window (e.g. N=15) yields zero jitter but introduces significant perceptual latency (sluggish response). N=7 provides an optimal compromise for 30 FPS webcam streams (~230ms lag).*

5. Stage 3: 5-Point Calibration & Screen Mapping

Objective: Convert physiological eye socket coordinates [0.0, 1.0] into actual user screen coordinates.

Why Calibration is Mathematically Obligatory:

When looking at the far corners of a laptop screen, the physical human iris only traverses a small subset of the eye socket (typically between 0.35 and 0.65). Without calibration, the user could never reach the screen boundaries. Calibration computes the exact **Gain (Amplification)** and **Offset (Center Bias)** for that user's posture and camera angle.

The 5 Calibration Target Coordinates:

Target Name	Target Screen Pos (X, Y)	User Action
1. Top-Left	(0.15, 0.15)	User fixes gaze on top-left bullseye for 30 frames
2. Top-Right	(0.85, 0.15)	User fixes gaze on top-right bullseye for 30 frames
3. Center	(0.50, 0.50)	User fixes gaze on center bullseye for 30 frames
4. Bottom-Left	(0.15, 0.85)	User fixes gaze on bottom-left bullseye for 30 frames
5. Bottom-Right	(0.85, 0.85)	User fixes gaze on bottom-right bullseye for 30 frames

Linear Least-Squares Regression (numpy.polyfit):

We fit 1st-degree polynomial equations relating raw eye coordinates to target screen coordinates:
   Screen_X = m_x * Gaze_X + c_x
   Screen_Y = m_y * Gaze_Y + c_y
Parameters are saved persistently to calibration.json so calibration persists across application restarts.

6. Remaining Pipeline Roadmap (Stages 4, 5, 6)

- **Stage 4 (Dwell Intent Detection):** Implements a continuous timer (1.8s) when calibrated gaze resides inside the left (0–15%) or right (85–100%) trigger zones to solve the *Midas Touch* problem.
- **Stage 5 (Presentation Control):** Connects PyAutoGUI to emit actual LEFT / RIGHT keyboard events, guarded by a safe DEMO_MODE flag.
- **Stage 6 (Logging & Evaluation):** Exports real frame-by-frame gaze coordinates to CSV (data/logs/gaze_log.csv) for quantitative error and latency plots.

7. Viva & Demonstration Q&A; Cheatsheet

Question from Evaluator	Strong Engineering Response
Why did you not use a Deep Learning classifier for gaze?	Custom neural networks require thousands of labeled user gaze images and high GPU compute. Our geometric pipeline (MediaPipe landmarks + relative normalization + linear calibration) is lightweight, runs in real-time on CPU at 30+ FPS, and provides explicit mathematical interpretability.
What is the 'Midas Touch' problem in gaze interaction?	The human eye is primarily a sensory organ, not an actuator. Everywhere the user looks cannot trigger a click. Dwell-time intent detection ensures only deliberate, sustained focus triggers a system command.
What are the real-world limitations of this system?	1. Variable room illumination alters eyelid contours. 2. Severe head rotations cause calibration drift. 3. Eyeglasses reflections can briefly occlude iris landmarks. 4. First-order linear mapping assumes flat planar projection rather than spherical eye geometry.